MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Rodion Bulimar FAF-242

Report

Laboratory Work 3
Cryptography

Checked by:
Zaica Maia, *university assistant*
FCIM, UTM

Chișinău – 2026

1 Purpose

The purpose of this laboratory work is to study polyalphabetic substitution ciphers and to implement the Playfair cipher (Task 3.1) for messages in Romanian, using an alphabet of 31 letters. The program lets the user choose between encryption and decryption, enter a key of at least 7 letters and a message or a ciphertext, validates the input, builds the 6×6 encryption matrix and prints the result.

2 Short Theory

Monoalphabetic ciphers keep the letter-frequency profile of the language, which makes them easy to attack. Polyalphabetic ciphers flatten this distribution because the substitution of a symbol depends on its position or context. The Playfair cipher, invented by Charles Wheatstone in 1854 and promoted by Lord Playfair, goes further: it encrypts *digrams* (pairs of letters) instead of single letters, so it is considered one of the first practical block ciphers.

Encryption has three stages:

1. **Text preparation.** The message is written in capital letters without spaces and split into pairs. If a pair would contain two identical letters, a filler letter (`X`) is inserted between them; if the last pair is incomplete, a filler is appended as well.
2. **Matrix construction.** The key is written row by row in the matrix, skipping repeated letters, then the remaining letters of the alphabet follow in alphabetical order. The Romanian alphabet has 31 letters, so a 6×6 matrix (36 cells) is used and the 5 free cells are filled with the digits `0`–`4`.
3. **Pair encryption.** For a pair (a, b) located at (ra, ca) and (rb, cb), with indices modulo 6:
   - same row: each letter is replaced by its right neighbour, c → (c + 1) mod 6;
   - same column: each letter is replaced by the one below, r → (r + 1) mod 6;
   - otherwise (rectangle): a → (ra, cb) and b → (rb, ca).

Decryption uses the same rectangle rule, while for the same row or column the shift is reversed (−1 instead of +1). The filler letters are removed manually after decryption, and the spaces are restored according to the meaning of the message.

3 Program Implementation

The program is written in Python (`lab_3.py`). The excerpts below show its main parts.

3.1 Alphabet, constants and input validation

```
ALFABET = "AĂÂBCDEFGHIÎJKLMNOPQRSȘTȚUVWXYZ"
SIMBOLURI_COMPLETARE = "01234"
SIMBOLURI_MATRICE = ALFABET + SIMBOLURI_COMPLETARE
RANDURI, COLOANE = 6, 6
LUNGIME_MINIMA_CHEIE = 7
CARACTER_COMPLETARE = "X"
CARACTER_COMPLETARE_ALTERNATIV = "Q"
```

```
def normalizeaza_text(text, nume="Text"):
    if not text or not text.strip():
        raise ValueError(f"{nume} nu poate fi gol.")
    if not re.fullmatch(r"[A-Za-z ]+", text):
        raise ValueError(... "Diapazon permis: 'A'-'Z' si 'a'-'z'")
    return text.replace(" ", "").upper()

def valideaza_cheie(cheie):
    cheie = normalizeaza_text(cheie, "Cheia")
    if len(cheie) < LUNGIME_MINIMA_CHEIE:
        raise ValueError(...)
    return cheie
```

Comment: The alphabet has exactly 31 letters (checked with `assert`); together with the 5 filler digits it fills the 6×6 matrix exactly. `normalizeaza_text` accepts only the characters A–Z and a–z (and spaces), removes spaces and converts the text to upper case; for any other character it raises an error that suggests the correct range. `valideaza_cheie` additionally requires at least 7 letters.

3.2 Matrix construction

```
def construieste_matrice(cheie):
    cheie = valideaza_cheie(cheie)
    simboluri, vazute = [], set()
    for caracter in cheie + ALFABET + SIMBOLURI_COMPLETARE:
        if caracter not in vazute:
            vazute.add(caracter)
            simboluri.append(caracter)
    return [simboluri[i*COLOANE:(i+1)*COLOANE] for i in range(RANDURI)]

def construieste_index(matrice):
    return {caracter: (rand, coloana)
            for rand, linie in enumerate(matrice)
            for coloana, caracter in enumerate(linie)}
```

Comment: The key, the alphabet and the filler digits are concatenated and duplicates are dropped while keeping the first occurrence. The resulting 36 symbols are cut into 6 rows. `construieste_index` builds a dictionary symbol → (row, column) so that positions are found in constant time.

3.3 Preparing the digrams

```
def pregateste_perechi(mesaj):
    text = normalizeaza_text(mesaj, "Mesajul")
    perechi, index = [], 0
    while index < len(text):
        primul = text[index]
        completare = "X" if primul != "X" else "Q"
        if index + 1 >= len(text):
            perechi.append(primul + completare); index += 1
        elif text[index] == text[index + 1]:
            perechi.append(primul + completare); index += 1
        else:
            perechi.append(text[index:index + 2]); index += 2
```

```
    return perechi
```

Comment: The text is scanned from left to right. If the two letters of a pair are equal, the first one is paired with a filler and the scan advances by one position only; an odd last letter is also completed with a filler. The filler is `X`, or `Q` when the letter itself is `X`, so that a pair of two identical letters never appears.

3.4 Pair transformation, encryption and decryption

```
def transforma_pereche(matrice, index, primul, al_doilea, directie):
    rand1, coloana1 = index[primul]
    rand2, coloana2 = index[al_doilea]
    if rand1 == rand2:
        return (matrice[rand1][(coloana1 + directie) % COLOANE]
              + matrice[rand2][(coloana2 + directie) % COLOANE])
    if coloana1 == coloana2:
        return (matrice[(rand1 + directie) % RANDURI][coloana1]
              + matrice[(rand2 + directie) % RANDURI][coloana2])
    return matrice[rand1][coloana2] + matrice[rand2][coloana1]
```

Comment: A single function implements all three Playfair rules. The parameter `directie` is +1 for encryption and −1 for decryption; the modulo operation gives the wrap-around at the edges of the matrix. The rectangle rule does not depend on the direction. `cripteaza` prepares the digrams and applies `transforma_pereche(..., 1)` to each of them, while `decripteaza` validates the ciphertext (allowed symbols: the 31 letters and the digits 0–4, even length) and applies `transforma_pereche(..., -1)`.

3.5 User interface

```
def citeste_valid(prompt, validator):
    while True:
        try:
            return validator(input(prompt))
        except ValueError as eroare:
            print(f"  [!] {eroare}")
```

Comment: `citeste_valid` repeats the question until the validator accepts the input, and prints the error message (with the correct character range) otherwise. `main()` shows a menu (1 – encryption, 2 – decryption, 0 – exit), reads the key, builds and prints the matrix, and then encrypts or decrypts. After decryption the program reminds the user to remove the filler letters and to restore the spaces manually, as required by the task.

4 Results

4.1 Worked example

Key: `MONARHIE`; message: *Per aspera ad astra*. The matrix built by the program is:

M	O	N	A	R	H

I	E	Ă	Â	B	C
D	F	G	Î	J	K
L	P	Q	S	Ș	T
Ț	U	V	W	X	Y
Z	0	1	2	3	4

The key letters occupy the first 8 cells, then the remaining letters of the alphabet follow; the last row contains the filler digits. The message is split into the pairs `PE RA SP ER AX AD AS TR AX` (the last `A` is completed with `X`). Checking several pairs by hand:

- `PE`: `P` (row 4, col. 2) and `E` (row 2, col. 2) are in the same column ⇒ the letters below: `U`, `F` ⇒ `UF`;
- `RA`: same row ⇒ right neighbours: `H`, `R` ⇒ `HR`;
- `SP`: `S` (row 4, col. 4) and `P` (row 4, col. 2) are in the same row ⇒ `Ș`, `Q` ⇒ `ȘQ`;
- `ER`: rectangle: `E` (row 2, col. 2), `R` (row 1, col. 5) ⇒ `B` (row 2, col. 5) and `O` (row 1, col. 2) ⇒ `BO`.

The program gives the ciphertext `UFHRȘQBORWMÎÂWȘHRW`, which coincides with the manual computation.

4.2 Program sessions

The following transcript shows a run of `python lab_3.py` (user input is written after the prompts). It illustrates the key-length validation, encryption, decryption and a second key.

```
============================================================
 Cifrul Playfair - alfabet romanesc (31 de litere)
============================================================
Cheia si mesajul accepta literele A-Z/a-z; spatiile se elimina.
Alege operatia (1 = criptare, 2 = decriptare, 0 = iesire): 1
Cheia (minimum 7 litere): abc
  [!] Cheia trebuie sa aiba cel putin 7 litere dupa eliminarea
      spatiilor (ai introdus 3).
Cheia (minimum 7 litere): MONARHIE

Matricea de criptare:
  M O N A R H
  I E Ă Â B C
  D F G Î J K
  L P Q S Ș T
  Ț U V W X Y
  Z 0 1 2 3 4

Mesajul de criptat: Per aspera ad astra
Mesaj pregatit (perechi): PE RA SP ER AX AD AS TR AX
Criptograma: UFHRȘQBORWMÎÂWȘHRW
Alege operatia (1 = criptare, 2 = decriptare, 0 = iesire): 2
Cheia (minimum 7 litere): MONARHIE
  (matricea ca mai sus)
Criptograma: UFHRȘQBORWMÎÂWȘHRW
Mesaj decriptat: PERASPERAXADASTRAX
(Elimina manual literele X/Q de completare si reintrodu spatiile
```

```
 conform sensului.)
Alege operatia (1 = criptare, 2 = decriptare, 0 = iesire): 1
Cheia (minimum 7 litere): CRIPTOGRAFIE

Matricea de criptare:
  C R I P T O
  G A F E Ă Â
  B D H Î J K
  L M N Q S Ș
  Ț U V W X Y
  Z 0 1 2 3 4

Mesajul de criptat: Mesaj secret ccaa
Mesaj pregatit (perechi): ME SA JS EC RE TC CA AX
Criptograma: QAMĂSXGPPAORRGĂU
Alege operatia (1 = criptare, 2 = decriptare, 0 = iesire): 2
Cheia (minimum 7 litere): CRIPTOGRAFIE
  (matricea ca mai sus)
Criptograma: QAMĂSXGPPAORRGĂU
Mesaj decriptat: MESAJSECRETCCAAX
Alege operatia (1 = criptare, 2 = decriptare, 0 = iesire): 0
La revedere!
```

Observations:

- the key `CRIPTOGRAFIE` contains repeated letters (`R`, `I`); they are written only once, so the key becomes `CRIPTOGAFE`;
- decryption returns exactly the prepared text (`PERASPERAXADASTRAX`); the filler `X` at the end must be removed and the spaces restored by hand: *PER ASPERA AD ASTRA*;
- in the second message the double letters `CC` fall across a pair boundary (`TC CA`), so no filler is inserted between them, while the odd last letter `A` receives the filler `X` (`AX`).

4.3 Input validation

Input	Field	Program reaction
`abc`	key	rejected: fewer than 7 letters
`Salut ă`	key / message	rejected: character outside A–Z, a–z
`12345678`	key	rejected: digits are not allowed
`(empty line)`	any	rejected: the field cannot be empty
`UFH`	ciphertext	rejected: even number of symbols required

5 Conclusions

The laboratory work allowed me to implement the Playfair cipher for the 31-letter Romanian alphabet using a 6×6 matrix completed with the digits 0–4. The encryption of digrams hides the frequency of single letters, so the cipher is stronger than monoalphabetic substitution, but it is still weak by modern standards because the frequency of digrams remains visible. A single function, `transforma_pereche`, handles encryption and decryption through the `directie` parameter. The implemented validation restricts the input to the characters A–Z and a–z, requires a key of at least

7 letters and reports the allowed range to the user. A limitation of the method is that the filler letters and the spaces are not restored automatically; this step is done manually, as required by the task.

6 Source Code

GitHub repository: https://github.com/RodionBm/Cryptolab3